



Lecture 7

AGENDA:

Gradient algorithms

Choice of learning rate

Methods of weights initialization

Network architecture

General, vector notation of training algorithm

$$\begin{aligned}\mathbf{W}(k+1) &= \mathbf{W}(k) + \Delta \mathbf{W}, \\ \Delta \mathbf{W} &= \eta \mathbf{p}(\mathbf{W})\end{aligned}$$

η - learning rate

$\mathbf{p}(\mathbf{W})$ – search direction in
multidimensional space $\mathbf{W}$

$$\mathbf{W} = [W_1, W_2, \dots, W_n]^T$$

Idea of the gradient algorithms

Gradient algorithms utilize approximation of target function $E(\mathbf{W})$ using Taylor's theorem on direction $\mathbf{p}$ around a given point $\mathbf{W}_0$ being the known solution

Taylor polynomial of $E(\mathbf{W})$ function

Let us assume:

$$\mathbf{g}(\mathbf{W}) = \nabla E(\mathbf{W}) - \text{gradient } E(\mathbf{W}),$$
$$\mathbf{H}(\mathbf{W}) - \text{Hessian } E(\mathbf{W})$$

Then:

$$E(\mathbf{W} + \mathbf{p}) = E(\mathbf{W}) + [\mathbf{g}(\mathbf{W})]^T \mathbf{p} + \frac{1}{2} \mathbf{p}^T \mathbf{H}(\mathbf{W}) \mathbf{p} + \dots$$

Taylor polynomial of $E(\mathbf{W})$ function

Where gradient:

$$\mathbf{g}(\mathbf{W}) = \nabla E(\mathbf{W}) = \left[\frac{\partial E}{\partial W_1}, \frac{\partial E}{\partial W_2}, \dots, \frac{\partial E}{\partial W_n} \right]^T$$

Hessjan:

$$H(W) = \begin{bmatrix} \frac{\partial^2 E}{\partial W_1 \partial W_1} & \dots & \frac{\partial^2 E}{\partial W_n \partial W_1} \\ \vdots & & \vdots \\ \frac{\partial^2 E}{\partial W_1 \partial W_n} & \dots & \frac{\partial^2 E}{\partial W_n \partial W_n} \end{bmatrix}$$

Simplification of Taylor polynomial of $E(\mathbf{W})$ function

$$E(\mathbf{W} + \mathbf{p}) = E(\mathbf{W}) + [\mathbf{g}(\mathbf{W})]^T \mathbf{p} + \frac{1}{2} \mathbf{p}^T \mathbf{H}(\mathbf{W}) \mathbf{p} + O(h^3)$$

$O(h^3)$ - approximation error



Point **$\mathbf{W}$** is the minimum of $E(\mathbf{W})$ function when:

- $\mathbf{g}(\mathbf{W}) = \mathbf{0}$ - there is an extremum

1D case:

First derivative equal to 0

- $\mathbf{H}(\mathbf{W})$ is positive – it describes the curvature of the function

1D case:

Second derivative is positive

Learning algorithms differences

Different methods of calculating:

- searching direction $\mathbf{p}$
- step η

Learning algorithm in pseudo-code

while (not MinimumFound) do

 Determine direction vector $\mathbf{p}$ in a point $\mathbf{W}$

 Decrease value of function $E(\mathbf{W})$ in direction $\mathbf{p}$

end while

Versions of gradient methods

- Steepest descent method (back propagation algorithm is implementation of this method)
- Variable metric method (utilizes information about curvature of the error function)
- Conjugate gradient method

Steepest descent

- Linear approximation of $E(\mathbf{W})$
- No information about curvature of error function delivered by Hessian
- In order to assure $E(\mathbf{W}_{k+1}) < E(\mathbf{W}_k)$

$$\mathbf{p} = -\mathbf{g}(\mathbf{W}),$$

therefore:

$$\Delta \mathbf{W} = -\eta \mathbf{g}(\mathbf{W})$$

Application of inertia for the steepest descend method

- For flat parts of the error function:

$$\Delta \mathbf{W}^{(l)} \approx \Delta \mathbf{W}^{(l+1)} = \Delta \mathbf{W}$$

therefore for inertial method with momentum:

$$\Delta \mathbf{W} = \eta \mathbf{p} + \mu \Delta \mathbf{W} \quad \Rightarrow \quad \Delta \mathbf{W} = \frac{1}{1-\mu} \eta \mathbf{p}$$

what gives $1/(1-\mu)$ times acceleration of the training process (for example $\mu=0.9$ gives 10 times acceleration)

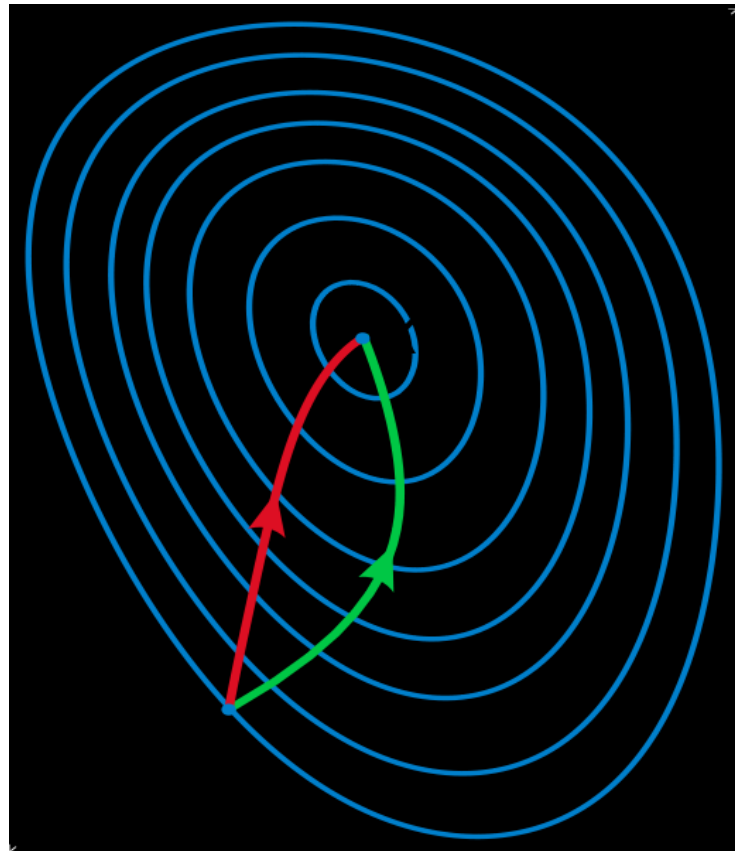
- For the steep parts of error function momentum suppresses oscillation

Variable metric method

- It is based on Newton's method of optimization
- It uses quadratic approximation of $E(\mathbf{W})$

$$E(\mathbf{W} + \mathbf{p}) \approx E(\mathbf{W}) + [\mathbf{g}(\mathbf{W})]^T \mathbf{p} + \frac{1}{2} \mathbf{p}^T \mathbf{H}(\mathbf{W}) \mathbf{p}$$

Variable metric method



Newton's method vs. steepest descent method

Variable metric method

- In the minimum of error function:

$$\frac{\partial E(\mathbf{W}_k + \mathbf{p}_k)}{\partial \mathbf{p}_k} = \mathbf{0}$$

therefore:

$$\mathbf{g}(\mathbf{W}_k) + \mathbf{H}(\mathbf{W}_k)\mathbf{p}_k = \mathbf{0}$$

and finally:

$$\mathbf{p}_k = -\mathbf{H}^{-1}(\mathbf{W}_k)\mathbf{g}(\mathbf{W}_k)$$

Variable metric method

- Calculating of the Hessian in each step of algorithm is too expensive.
- Therefore, the approximated Hessian $\mathbf{G}(\mathbf{W})$ is used.
- $\mathbf{G}(\mathbf{W})$ is modified to assure the proper approximation of E function curvature according to the equation:

$$\mathbf{G}(\mathbf{W}_k)(\mathbf{W}_k - \mathbf{W}_{k-1}) = \mathbf{g}(\mathbf{W}_k) - \mathbf{g}(\mathbf{W}_{k-1})$$

Variable metric method

- Instead of calculating $\mathbf{G}(\mathbf{W}_k)$ it is more advantageous to designate $\mathbf{V}_k = \mathbf{G}^{-1}(\mathbf{W}_k)$ what is approximation of inversion of the Hessian
- According to the BFGS method it is equal to:

$$\mathbf{V}_k = \mathbf{V}_{k-1} + \left(1 + \frac{\mathbf{r}_k^T \mathbf{V}_{k-1} \mathbf{r}_k}{\mathbf{s}_k^T \mathbf{r}_k} \right) \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{s}_k^T \mathbf{r}_k} - \frac{\mathbf{s}_k \mathbf{r}_k^T \mathbf{V}_{k-1} + \mathbf{V}_{k-1} \mathbf{r}_k \mathbf{s}_k^T}{\mathbf{s}_k^T \mathbf{r}_k}$$

$$\mathbf{V}_0 = \mathbf{I}$$

where:

$$\mathbf{s}_k = \mathbf{W}_k - \mathbf{W}_{k-1}, \quad \mathbf{r}_k = \mathbf{g}(\mathbf{W}_k) - \mathbf{g}(\mathbf{W}_{k-1})$$

Variable metric method

In the DFP method, $\mathbf{V}_k$ is calculated as follows:

$$\mathbf{V}_k = \mathbf{V}_{k-1} + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{s}_k^T \mathbf{r}_k} - \frac{\mathbf{V}_{k-1} \mathbf{r}_k \mathbf{r}_k^T \mathbf{V}_{k-1}}{\mathbf{r}_k^T \mathbf{V}_{k-1} \mathbf{r}_k}$$

$$\mathbf{V}_0 = \mathbf{1}$$

Variable metric method

When the $\mathbf{V}_k$ value is calculated, the $\mathbf{p}_k$ can be set:

$$\mathbf{p}_k = -\mathbf{V}_k \mathbf{g}(\mathbf{W}_k)$$

And the vector of weights change $\Delta\mathbf{W}_k$ as:

$$\Delta\mathbf{W}_k = \eta \mathbf{p}_k$$

Variable metric method - remarks

- For the first iteration the steepest descend method must be applied because $\mathbf{V}_0 = \mathbf{1}$
- For the calculated $\mathbf{p}_k$ it is necessary to perform directional minimalization
- Method is considered as one of the best multivariable optimization methods
- Disadvantage is big calculation and memory complexity (a lot of Hessian elements). It is applied typically for relatively small networks (number of weights < 1000)

Conjugate gradient method

- The curvature information contained in Hessian is not utilized
- The subsequent steps of minimalization $\mathbf{p}_k$ are set to be orthogonal and conjugate with all other directional vectors $\mathbf{p}_0, \dots, \mathbf{p}_{k-1}$
- Set of $\mathbf{p}_i$ vectors, where $i=0, \dots, k$ is conjugate in respect to the $\mathbf{G}$ matrix if the following equation is satisfied $\mathbf{p}_i^T \mathbf{G} \mathbf{p}_j = 0$ for $i \neq j$.

Conjugate gradient method

$\mathbf{p}_k$ vector that satisfies the orthogonality assumptions can be formulated using the following recurrent equation:

$$\mathbf{p}_k = -\mathbf{g}_k + \beta_{k-1}\mathbf{p}_{k-1}$$

Value of $\mathbf{p}_k$ depends only on current gradient $\mathbf{g}_k$, previous direction $\mathbf{p}_{k-1}$ and conjugate factor β_{k-1} that cumulates information about all previous directions

Conjugate gradient method

The value of conjugate factor can be calculated according to:

$$\beta_{k-1} = \frac{\mathbf{g}_k^T (\mathbf{g}_k - \mathbf{g}_{k-1})}{\mathbf{g}_{k-1}^T \mathbf{g}_{k-1}}$$

or:

$$\beta_{k-1} = \frac{\mathbf{g}_k^T (\mathbf{g}_k - \mathbf{g}_{k-1})}{-\mathbf{p}_{k-1} \mathbf{g}_{k-1}}$$

Conjugate gradient method

Direction of $\mathbf{p}_k$ in the initial step ($k = 0$) must be calculated using the steepest descend method because the value of β_{k-1} (conjugate factor) is not known

Conjugate gradient method

- After n steps (where n is the number of weights) “history is cleared” and algorithm restarts using the steepest descend method
- Directional minimization is recommended but not obligatory

Conjugate gradient method - summary

- Typically, it is used for big networks.
- It is considered as the faster method for big nets.

Choice of learning rate

- Constant rate for all layers(ie. $\eta = 0.05$, $\mu = 0.9$)
- Constant rates (yet different for different layers)

$$\eta \leq \min(1/n_i)$$

where: n_i is the number of inputs of i -th neuron in the layer considered

- These methods are simple but not very efficient

Choice of learning rate

Adaptive method (if the error decreases, then increase η). Let error ε be defined as:

$$\varepsilon = \sqrt{\sum_{j=1}^M (y_j - d_j)^2}$$

If $\varepsilon_i > k_w \varepsilon_{i-1}$ then: $\eta_{i+1} = \eta_i \rho_d$ ($\rho_d < 1$)

else: $\eta_{i+1} = \eta_i \rho_i$ ($\rho_i > 1$)

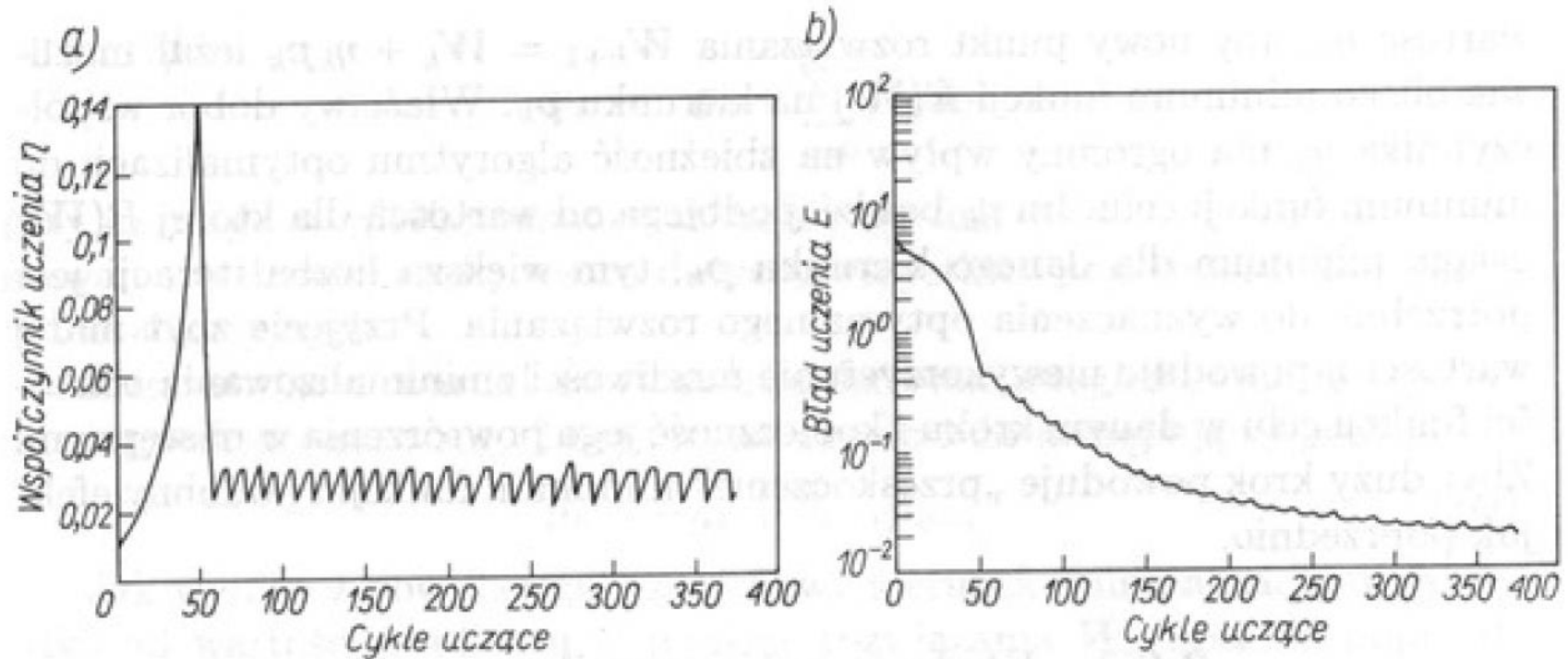
Example:

Error growth acceptance factor: $k_w = 1.04$,

Decreasing factor η : $\rho_d = 0.7$,

Increasing factor η : $\rho_i = 1.05$)

Choice of learning rate



Choice of learning rate

Directional minimization:

The rate η should be chosen in such a way that the point $\mathbf{W}_{k+1} = \mathbf{W}_k + \eta_k \mathbf{p}_k$ be the minimum of the function $E(\mathbf{W})$ in direction $\mathbf{p}_k$.

Methods of weights initialization

- Weights should be initialized properly
- Since it is unknown what values the „proper” weights should have, it is advantageous to use multistart procedure

Methods of weights initialization

- Random initialization
- Unsupervised learning:
 - with the use of Hebb's rule
 - with the self-organization algorithm

Random initialization

Typical range for random variables values:

$$\left[-\frac{a}{\sqrt{n_{in}}}, \frac{a}{\sqrt{n_{in}}} \right]$$

For sigmoid function: $a = 2.38$,
 n_{in} – number of inputs of neuron

Random initialization, cont.

Based on an analogy with the dynamics of spin glass, the following range has been proposed:

$$\begin{cases} \left[-\frac{\sqrt{n_{in}}}{2}, \frac{\sqrt{n_{in}}}{2} \right] & \text{for hidden neurons} \\ 0, & \text{for output neurons} \end{cases}$$

Network architecture

- Maximum two hidden layers of fully connected layers
- Kolmogorow theorem: For arbitrary approximation of the scalar continuous function it is enough to use one hidden layer of the size $2N+1$ (where N denotes the number of inputs to the network)
- However, this number is not optimal: usually, it is better to use less neurons: for example $\sqrt{NM}$ (where M denotes the number of outputs) or even less.

Problem with architecture

- To many weights (number of weights is a function of the number of hidden neurons) results in poor generalisation (there are too many degrees of freedom which cannot be fixed by training examples)
- Too less weights results in poor learning (there is too less degrees of freedom for appropriate approximation of the problem under consideration)
- Theoretically, the number of learning facts should be around 20 times larger than the number of weights – it is very hard to achieve in practice

Solutions

- Reduction, while started with large network.
 - OBD (optimal brain damage), OBS (optimal brain surgeon) – based on sensitivity analysis
 - Penalty function (for large values of weights it tries to reduce them and possibly eliminate when they become sufficiently close to zero)
- Addition of neurons (start with small number of neurons)
Fahlman cascade correlation algorithm

Other methods for increasing the generalization

- Neural networks with functional extension (Pao networks).
- Motivation: for badly posed problems ($N \ll M$) it is possible to use additional inputs as functions of basic inputs (projection into space with more dimensions helps by simplification of architecture and better generalization)
- Example: using input x_1 , x_2 and x_1x_2 it is possible to solve XOR problem without hidden layer

Better generalization by a proper choice of data

- Analysis of training data
- Adding noise to training data
 - data augmentation



Thank you